In this manual

1. Overview and included content
2. Installation and first successful play
3. Folder structure and source reference
4. Nut Bolt Sort reskin: interface colours and visible text
5. Gameplay
6. Level editing
7. Generation
8. Boosters
9. LiveOps and ads
10. Builds
11. Troubleshooting
12. Artwork, fonts and audio replacement
13. Level editor: concrete editing workflow
14. Procedural generation and level data reference
15. Authoring controls and booster configuration
16. Game text versus saved data and service configuration
17. Reskin troubleshooting and release check
18. Licences, support and documentation access

Nut Bolt Sort

Developer manual · Version 1.0.1 · SatisVizion · Documentation revised 26 October 2026

Overview and included content

Tap a source bolt to lift its matching top nuts, then tap a compatible destination bolt. Group every colour together to clear the bench.

This is an editable Unity game template and starting point for your own title. Customize the visuals, copy, level progression and release settings for your audience, then test your changes. The included Android demo is a showcase build; it is not a signing key or a production store submission.

URP procedural workshop meshes with uGUI; tap-based sort puzzle The campaign contains 25 authored levels and game-specific procedural continuation. The included service kit is HyperCasual LiveOps Kit. Ads ship in Dummy/simulated mode and IAP is Disabled; third-party advertising/billing SDKs are not bundled.

Installation and first successful play

1. Back up your project and extract the main ZIP completely. Read `Documentation/Documentation.html` before importing the package. Offline documents do not require the hosted page to be available.
2. Install Unity `6000.0.62f1` through Unity Hub. Add Android Build Support (including the matching SDK,

NDK and OpenJDK) if building Android, and WebGL Build Support if building browser demos. A supplied source project can be opened with Add project in Unity Hub.

3. For package import into a new project, select a Universal Render Pipeline template. Use `Assets > Import Package > Custom Package` and import the complete package. Do not omit the private game Kit, HyperCasualLiveOpsKit, fonts, settings, scenes or their .meta files.
4. Check `Window > Package Manager`: Universal RP `17.0.4`, Unity UI `2.0.0` and Input System `1.14.2`. Standard Unity modules include `com.unity.modules.screencapture` and `com.unity.modules.androidjni`. Let Unity finish compilation before opening a scene.
5. Set `Edit > Project Settings > Player > Other Settings > Active Input Handling` to `Both` manually. Restart if Unity requests it. The documented setup is a user action; do not rely on an import hook changing a buyer's Project Settings.
6. In `Edit > Project Settings > Graphics` select the supplied URP asset, and check the pipeline assignment for the active Quality level. Retain the packaged Settings assets and GUID references. A pink scene usually indicates a missing pipeline or shader assignment.
7. Open `Assets/Games/NutSort/Scenes/NutSort_Mobile.unity`. Press Play, focus the Game view, and follow the first-level tutorial: Tap a source bolt to lift its matching top nuts, then tap a compatible destination bolt. Group every colour together to clear the bench.
8. Stop Play mode before editing source or authored content. Open the landscape scene `Assets/Games/NutSort/Scenes/NutSort_WebGL.unity` and verify the alternate layout as well. Runtime-created UI changes made only in Play mode are not a saved reskin.

Folder structure and source reference

The concrete paths in this manual match this standalone/Codester layout. The game root is `Assets/Games/NutSort`. Keep .meta files with their assets so scene, font, sprite and material references remain valid.

```
Assets/Games/NutSort/
  Scenes/            Mobile and landscape entry scenes
  Scripts/           Rules, HUD, board view and game data
  Editor/            Game-specific authoring tools
  Resources/         Level JSON, artwork and game audio
  Kit/               Required game runtime and editor helpers
  Documentation/     This manual
Assets/HyperCasualLiveOpsKit/        Included service kit
Assets/TextMesh Pro/              Required text resources
```

Exact file	What to edit or inspect
`Assets/Games/NutSort/Scripts/NsGen.cs`	Procedural level creation and seed/difficulty rules.
`Assets/Games/NutSort/Scripts/NsLevels.cs`	Campaign data loading, level access and procedural continuation.
`Assets/Games/NutSort/Scripts/NsMeshes.cs`	Procedural mesh construction.
`Assets/Games/NutSort/Scripts/NsModel.cs`	Logical board representation and rule state.
`Assets/Games/NutSort/Scripts/NsSfx.cs`	Game-specific sound event helpers.
`Assets/Games/NutSort/Scripts/NutSortGame.cs`	Main game lifecycle, progression, input/rule orchestration and booster handlers.
`Assets/Games/NutSort/Scripts/NutSortGame.Hud.cs`	Runtime HUD, menu and popup construction, display strings and tutorial presentation.
`Assets/Games/NutSort/Scripts/NutSortGame.View.cs`	Scene geometry or board rendering, camera layout and visual feedback.
`Assets/Games/NutSort/Kit/Runtime/HzUI.cs`	Creates the uGUI labels/buttons and resolves the game kit fonts.

`Assets/Games/NutSort/Kit/Runtime/HzAudio.cs`	Named sound resources, music playback where provided and fallback sound synthesis.
`Assets/Games/NutSort/Kit/Runtime/HzCore.cs`	Game-prefixed local persistence helpers.
`Assets/Games/NutSort/Kit/Runtime/HzCore.cs`	Bridge from game calls to simulated/service-kit ads and native haptic hooks.
`Assets/Games/NutSort/Editor/NutSortLevelEditor.cs`	The visual level editor and its actual menu commands.

Nut Bolt Sort reskin: interface colours and visible text

The interface is constructed in C# when the scene runs. There is no single static scene prefab containing every final label. Use the exact game files below; changing only a shared-kit theme or localization table will not replace game-authored HUD/menu strings.

1. Change interface colours

Open `Assets/Games/NutSort/Scripts/NutSortGame.Hud.cs` in your C# editor. Find these declared palette symbols; change only the quoted hexadecimal value in the existing `HzUI.Hex(...)` assignment. Preserve the symbol, type and surrounding syntax. This keeps every caller using the shared colour in sync.

File / symbol	Current value	Concrete usage to inspect
`Scripts/NutSortGame.Hud.cs Orange`	`#FF8A2B`	`NutSortGame.Hud.cs: var p = HzUI.Panel("Banner", _col, Orange, 40);`
`Scripts/NutSortGame.Hud.cs Blue`	`#2F8CFF`	`NutSortGame.Hud.cs: var settings = HzIcons.IconButton(_col, HzIcons.Settings, Blue, () => { HzAudio.Play(Sfx.Select, 0.6f); ShowSettingsPopup(); }, 128);`
`Scripts/NutSortGame.Hud.cs Navy`	`#1B1F4B`	Palette helper in `NutSortGame.Hud.cs`; search `Navy` in its call sites, including generated visuals.
`Scripts/NutSortGame.Hud.cs Brass`	`#F2B233`	`NutSortGame.Hud.cs: var title = HzUI.Label("Title", left, "NUT BOLT\nSORT", (int)(78 * s), Brass);`
`Scripts/NutSortGame.Hud.cs Cyan`	`#3EC6FF`	`NutSortGame.Hud.cs: var h = HzUI.Label("H", card.transform, "WORKBENCH", (int)(40 * s), Cyan); h.rectTransform.Place(new Vector2(0.5f, 1), new Vector2(400, 60 * s), new V`

Concrete first colour edit: in `Assets/Games/NutSort/Scripts/NutSortGame.Hud.cs`, find this fragment within its existing declaration:

```
Navy = HzUI.Hex("#1B1F4B")
```

Replace only that colour value, for example:

```
Navy = HzUI.Hex("#24485C")
```

1. Save the source and return to Unity. Wait for compilation, resolve any Console errors, and start a fresh Play session so the UI and cached generated artwork are rebuilt.
2. Inspect the changed HUD surface, settings menu and a win/lose or booster popup. Verify that text remains readable, selected/disabled states remain distinct, and icons are visible against the new background.

3. Check the mobile and landscape scenes. Some popup, world, camera and material colours are separate literal assignments in `Scripts/NutSortGame.Hud.cs` or the .View.cs file. If one surface did not change, inspect the colour argument used by that exact control instead of replacing every matching hex value globally.
4. For a procedural colour change, stop and re-enter Play; if you deliberately disabled domain reload, restart the editor when testing static sprite/material caches. A bitmap with baked-in colours must be edited separately from the C# palette.

2. Change a visible HUD label

Open `Assets/Games/NutSort/Scripts/NutSortGame.Hud.cs`, find `BuildHud` and locate this exact source line:

```
_levelText = HzUI.Label("Level", _col, "LEVEL " + Level, 86, Color.white);
```

To call levels stages, change the display string while preserving the level expression and formatting:

```
_levelText = HzUI.Label("Level", _col, "Stage " + Level, 86, Color.white);
```

This is a targeted edit to the displayed string, not a global search/replace of object names. `HzUI.Label("Lvl", parent, "LEVEL " + Level, ...)` uses its first string as a hierarchy/object name and its third argument as visible text. In assignments such as `_levelTxt.text = ...`, edit only the literal part; retain the number, interpolation braces, arithmetic, conditional branches and formatting.

3. Edit menus, popups, title and booster captions

Use this game-specific lookup table. The snippet is a search target in the named file; it shows the actual display-writing call rather than an assumed localization key. Line numbers are navigation hints for this revision and can shift after your edits.

Exact file and method	Search for this source
`Scripts/NutSortGame.Hud.cs` `BuildHud` line 113	`_levelText = HzUI.Label("Level", _col, "LEVEL " + Level, 86, Color.white);`
`Scripts/NutSortGame.Hud.cs` `ShowSettingsPopup` line 655	`var p = WorkshopPopup("Settings", "", null,`
`Scripts/NutSortGame.Hud.cs` `ShowHowTo` line 671	`WorkshopPopup("How to play",`
`Scripts/NutSortGame.cs` class declarations line 21	`public override string Title => "Nut Bolt Sort";`
`Scripts/NutSortGame.cs` `WinFlow` line 285	`WorkshopPopup(lv <= 2 ? "Nicely done!" : _def.hard ? "Hard level beaten!" : "Bench cleared!",`
`Scripts/NutSortGame.cs` `LoseFlow` line 307	`WorkshopPopup("Out of moves!", "No nut can move.\nTake a step back or add a spare bolt.", Art("lose_badge"), buttons.ToArray());`

`Scripts/NutSortGame.Hud.cs BuildSides` line 202	`var title = HzUI.Label("Title", left, "NUT BOLT\nSORT", (int)(78 * s), Brass);`

1. Change the `Title` property in `Scripts/NutSortGame.cs` for the game title exposed by the base game. Also change any separate literal title built by the HUD/view; a Title property is not a universal replacement for all title art.
2. Change visible menu captions in `ShowSettingsPopup` and the help-page strings in `NutSortGame.Hud.cs`. Keep action callbacks, toggle variables and control object names unchanged.
3. For booster names edit `BoostName` in `Assets/Games/NutSort/Scripts/NutSortGame.cs`. For booster help edit `BoostDesc` in `Assets/Games/NutSort/Scripts/NutSortGame.cs`. Preserve array lengths/order, booster IDs, unlock logic and charge values. A cosmetic caption must not rename an ad placement, resource lookup or PlayerPrefs key.
4. For status counters, change the actual refresh method, not only the label initializer; otherwise the next frame or level load may overwrite your new caption. Update singular/plural and portrait/landscape variants together.
5. Translate tutorial/help/result text in the game files and level `name` / `hint` fields where used. Keep `\n` line breaks and `{...}` interpolations valid, and test long lines in narrow portrait popups.

4. What the shared localization guide does and does not cover

The shared HyperCasual LiveOps Kit localization guide concerns UI and content that use that kit's localization service. This game's literal strings in the files above are not automatically replaced by adding entries to an HLK table. For one-language reskinning, edit the literals and data shown here. For runtime language selection, explicitly route those game display assignments through your chosen lookup service, provide keys for each caption, and refresh active labels when the language changes. That is an integration task, not a delivered translation toggle.

The landscape title is the two-line string NUT BOLT\nSORT in BuildSides; the portrait level heading is created in BuildHud. Both live in NutSortGame.Hud.cs. Changing Title in NutSortGame.cs alone does not replace that landscape title. Booster display names are in BoostName. Nut colours and material/mesh details are in the view and material-generation code and are separate from the Orange/Blue/Navy/Brass/Cyan HUD constants.

5. Verify the reskin

1. Launch both supplied scenes with a normal player save. Confirm the edited level heading, game title, settings/help buttons, counters and booster names.
2. Open a result popup and a refill offer, and replay the first-level and first-booster tutorials using a disposable test profile or the documented developer controls. Confirm that the changed colours remain readable behind all overlays.
3. Test one valid move, one invalid action, progression to the next level, a retry and one available booster. A text reskin must not change goals, costs, rule checks or resource IDs.
4. Make a new Android/WebGL test build after source edits. Existing hosted demos and videos will continue to show their previously built appearance until rebuilt; editing this manual alone does not change those binaries.

Gameplay

Click or tap a bolt, then another bolt. Only a matching top colour or an empty bolt can receive a move; available space limits how many nuts transfer. Filled single-colour bolts receive a cap. Rust hides colours until exposed, a

key nut opens a padlocked bolt when moved away, and tall bolts hold five nuts.

The first levels guide the basic moves. New mechanics and boosters introduce themselves through popups. Completion awards gears: 20 for a normal level or 40 for a hard level, plus any other earned gears. Progress, charges and settings save locally through PlayerPrefs; WebGL storage depends on the browser profile. When no legal move remains, the recovery popup offers applicable undo/spare/retry actions.

The source renders its workshop, bolts and nuts using procedural meshes and URP materials, with uGUI overlays. The interaction is a tap-based sorting puzzle. Use the supplied scene and camera setup for its workshop presentation.

Level editing

Open **Nut Bolt Sort > Level Editor**. The editor reads and writes `Assets/Games/NutSort/Resources/nutsort/Levels/levels.json`.

To make a first edit:

1. Select an existing level in the list and duplicate it, keeping the original tutorial unchanged.
2. Select a bolt, choose a nut colour, and paint the bolt from bottom to top. Use the erase brush to remove that slot and the nuts above it. Keep colour totals consistent with the bolt capacity.
3. Adjust bolt capacity, rust, key position and the optional padlocked bolt. The key must be on a different bolt from the lock.
4. Run the editor's validation/solver controls. Read the actual result; a design difficulty value is a target, not a proof that a player will solve it.
5. Save the level file. In Play mode open the **DEV** panel and jump to the new level number. Play the changed board and confirm its intended solution.

The editor supports add, duplicate, delete, reorder, undo and save prompts. **Nut Bolt Sort → Validate All Levels** is available for authoring validation.

Data format

`bolts` strings read **bottom to top**. Lowercase `a`–`l` are colours; uppercase marks rust. `cap` is bolt capacity. `lockBolt`, `keyBolt` and `keySlot` are zero-based; `-1` disables the lock/key. `hard`, `breather`, `tut`, `hint` and `difficulty` control presentation, guidance and generation intent.

```
{
  "version": 1,
  "levels": [
    {
      "name": "First Turn",
      "cap": 4,
      "bolts": [
        "aaa",
        "a"
      ],
      "lockBolt": -1,
      "keyBolt": -1,
      "keySlot": -1,
      "hard": false,
      "breather": true,
      "tut": 1,
      "hint": "",
      "difficulty": 0
    }
  ]
}
```

The top nut is normalised clean by the runtime. A winning bolt is full and all one colour. Keep the model's maximums in mind: 12 colours, 16 bolts and capacity up to 6; the delivered design uses four- and five-nut bolts.

Generation

The editor's **Auto Generate** controls include count/start, seed, empty bolts, colour range, rust range, difficulty range, breathers, locks, tall bolts, spread layout and replacement. Generate a small batch, inspect its solver/difficulty results, save, then play the new boards.

After the 25 authored levels, `NsLevels.Get` uses `NsGen.Curve` and the deterministic seed `level * 7919 + 1301`. Generated levels cycle through breather, rust, lock, combined and hard recipes. The generator uses bounded solver attempts, alternate seeds, extra room and a constructive fallback. The delivered content is **25 authored levels plus procedural continuation**.

Boosters

Booster	Unlock	Initial charges	Effect
Undo	3	3	Reverses a recorded move.
Hint	5	2	Shows a suggested next move.
Spare Bolt	8	2	Adds one empty bolt, once per level and within the model's bolt limit.
Rust Spray	10	2	Choose a rusty bolt to clean its rust.

When a usable booster has no charges, its popup offers one charge for **60 gears** when affordable or a rewarded-ad refill. A reward is granted only when the ad callback reports completion. Failed/skipped/unavailable ads do not grant a reward. Tutorial guidance can temporarily restrict unrelated actions.

LiveOps and ads

Included LiveOps asset: HyperCasual LiveOps Kit. Its complete offline manual is included alongside the game.

Configuration: `Assets/HyperCasualLiveOpsKit/Resources/HLK/HLKSettings.asset`. The delivered configuration uses **Ads Mode Dummy**, with simulated banner, interstitial and rewarded ads and **no advertising provider SDK**. Integration is via `Kit/Runtime/HzAds.cs`, `HzPlatform`, and the three placement assets under `Kit/Resources/nutsort/kit/Ads`.

The banner reserves layout space. Interstitials occur at natural progression breaks. Rewarded ads support booster refill and eligible recovery actions. These are simulation features for development/showcase, not live revenue-generating ads. Public sample ad-unit identifiers, where present, are test identifiers. Native haptics require an appropriate device; browser behaviour differs.

No advertising provider SDK or External Dependency Manager is bundled. IAP is Disabled (mode 3); no billing SDK is bundled. Automatic provider installers are disabled. If you choose to enable a provider, install it separately from the provider's official distribution, follow the HLK manual, configure your own identifiers/consent settings, and test the selected platform. Do not enable SDK scripting symbols for packages you have not installed. Other HLK services are configurable kit capabilities; the game does not claim every kit service is configured for production.

Builds

Android: select the Mobile scene, Android build target, portrait orientation, your own application identifier,

ARM64/IL2CPP and matching Unity Android tools. The supplied demo is a development APK signed with a debug key. It is not a signed production AAB or a Google Play submission. Configure your own signing and release settings for publication.

WebGL: select the Mobile scene for the portrait demo, the Default WebGL template, and build with Unity's WebGL module. Serve the entire output over HTTP(S). Keep loader, framework, data and WASM files together; opening `index.html` via `file://` is unsupported. Brotli with Unity decompression fallback is suitable for a static host. Browser audio normally needs the first click/tap. Native services and haptics have browser limitations.

For the supplied standalone project's two-scene landscape-first route, `?mobile=1` switches to the following Mobile scene. A portrait-only build should simply use the Mobile scene without that switch.

Prepare your Android release

1. Keep an unchanged backup. Open **Edit → Project Settings → Player**. Change Company Name, Product Name and Version to your own values. Set the Android application identifier under Other Settings to a unique reverse-domain identifier you control; do not publish using `com.satisvizion.nutsort.demo`. Increase Bundle Version Code for each uploaded Android update.
2. Under Player → Icon, assign your own default icon and Android adaptive foreground/background layers. Inspect all required sizes. Configure Splash Image and the app's visible name for your brand. Change in-game title/artwork in the locations listed in Customisation. Test text, contrast and touch targets on a real phone.
3. Install the Android module, SDK/NDK and OpenJDK through the same Unity Hub editor installation. Select Android, ARM64 and IL2CPP, portrait orientation, and the Mobile scene. Review minimum/target API requirements against the current destination store. Turn off Development Build, Script Debugging and Autoconnect Profiler for a release.
4. In Player → Publishing Settings, create or select **your own keystore and key alias**. Keep the keystore, alias and passwords in a secure backup; retain them for future updates. The supplied APK uses a debug key and cannot serve as your production signing identity. Never distribute your signing secrets with game source.
5. For Google Play, enable Build App Bundle and build an AAB with your release signing configuration. For direct device testing, build an APK. Install and play on a real Android device, including pause/resume, audio, touch, haptics and saved progression, before submission. This delivery includes a compiled Android demo; no physical-device installation test is claimed.
6. The delivered Ads mode is Dummy and IAP is Disabled. To add real services, separately install the official provider SDK you choose, follow the included HLK manual, and configure your own application/ad-unit/product identifiers and consent flow. Test rewarded completion, cancellation, unavailable inventory and network failure. Provide a privacy policy matching the actual services you enable and complete the destination store's data/ads declarations accurately.
7. Use the global DEV panel during authoring. Add `HZ_RELEASE` to Player scripting defines when you want it excluded from a consumer build. This symbol changes development controls; it does not configure ad providers or certify production readiness.

Responsive WebGL player

After building, use **Nut Bolt Sort → Create Responsive WebGL Page** and select the build output folder. This uses the included `Editor/WebGLPlayer.html` to generate an original responsive page around the completed Unity build. Preserve the complete Build folder. For a portrait-only build use the Mobile scene. In a landscape-first two-scene build, the supported `?mobile=1` route selects the following Mobile scene. Keep Brotli decompression fallback enabled on a static host without custom content-encoding rules.

The game's WebGL build menu uses the kit's non-development local-showcase build scope. This is suitable for a public simulated demo; it does not enable production mobile services. For a production service build, use

HLK's build doctor and resolve the checks for your chosen providers. A browser build has different native-service capabilities from Android.

Troubleshooting

- **Missing types after import:** install the documented Unity packages, import all three required roots, and keep `.meta` files.
- **Pointer input exceptions:** set Active Input Handling to Both and restart the editor.
- **Pink materials or wrong rendering:** use URP 17.0.4 and the supplied NutSort pipeline/renderer; run Setup and select the built-in WebGL template.
- **No real ad revenue:** Dummy mode is simulated. A separately configured provider and its SDK are required for real ads.
- **A booster is unavailable:** check its unlock level, charges, current tutorial/guide, and whether its action is applicable.
- **An imported scene appears sparse in edit mode:** UI and workshop objects are built at runtime. Press Play.
- **Progress persists between tests:** use the developer reset controls; clearing PlayerPrefs removes saved progress/settings.
- **An external URL is unavailable offline:** this manual and the included kit manual are local HTML. External provider links require internet.

Artwork, fonts and audio replacement

Game-specific resource directories in this edition:

Path	Purpose
`Assets/Games/NutSort/Resources/nutsort/Art`	Art assets loaded by the game; keep resource base names stable unless you update every lookup.
`Assets/Games/NutSort/Resources/nutsort/Levels`	Levels assets loaded by the game; keep resource base names stable unless you update every lookup.
`Assets/Games/NutSort/Resources/nutsort/Materials`	Materials assets loaded by the game; keep resource base names stable unless you update every lookup.
`Assets/Games/NutSort/Resources/nutsort/Music`	Music assets loaded by the game; keep resource base names stable unless you update every lookup.
`Assets/Games/NutSort/Resources/nutsort/Sfx`	Sfx assets loaded by the game; keep resource base names stable unless you update every lookup.

1. Replace an existing sprite or audio clip through Unity's Project view while keeping its resource base name and .meta/GUID. If you rename or move it, update its `Resources.Load` lookup and any serialized references together.
2. For a UI icon, find the `Art(...)` / art-class `Get(...)` call in the HUD script and its base name under the Art directory. Imported sprite colours may be baked into the pixels; palette edits only affect code-applied tint or generated geometry.
3. For a 3D scene, inspect the `.View.cs` file and any game-specific material/import helper before rerunning material generation. Back up custom materials first; generation commands may overwrite generated material settings.
4. The game kit resolves its fonts in `Assets/Games/NutSort/Kit/Runtime/HzUI.cs`. Read `Display` and `Body` for

the current resource paths. Use fonts with redistribution rights, retain their licence notices, and test accented characters/glyph coverage at the existing sizes.

5. For sound effects, inspect the game's sound-name constants/calls and `Assets/Games/NutSort/Kit/Runtime/HzAudio.cs`. Replace the matching Sfx resource without changing its base name. Named event variants and synthesized fallbacks are handled by the kit; a new filename does nothing unless a caller requests it.
6. Music is a separate resource/playback path. Match the `PlayMusic` lookup if this game supplies a track. Set reasonable clip/volume settings and test mute, focus/background behaviour and device output. Do not infer audio-device testing from a successful build.

Retain the original licence/attribution files for fonts, icons, audio and other included dependencies when distributing your customized game.

Level editor: concrete editing workflow

Open `Nut Bolt Sort > Level Editor`. Its implementation is `Assets/Games/NutSort/Editor/NutSortLevelEditor.cs`. The following names are taken from that editor, so use the displayed control name rather than assuming a generic inspector.

Control / field	Practical use
`Reload`	Select the relevant level first; read the adjacent controls/help in the editor before applying it. Save only after validating the changed content.
`Undo`	Select the relevant level first; read the adjacent controls/help in the editor before applying it. Save only after validating the changed content.
`Redo`	Select the relevant level first; read the adjacent controls/help in the editor before applying it. Save only after validating the changed content.
`Validate`	Select the relevant level first; read the adjacent controls/help in the editor before applying it. Save only after validating the changed content.
`Validate All`	Select the relevant level first; read the adjacent controls/help in the editor before applying it. Save only after validating the changed content.
`Add`	Select the relevant level first; read the adjacent controls/help in the editor before applying it. Save only after validating the changed content.
`Dup`	Select the relevant level first; read the adjacent controls/help in the editor before applying it. Save only after validating the changed content.
`Del`	Select the relevant level first; read the adjacent controls/help in the editor before applying it. Save only after validating the changed content.
`Up`	Select the relevant level first; read the adjacent controls/help in the editor before applying it. Save only after validating the changed content.
`Down`	Select the relevant level first; read the adjacent controls/help in the editor before applying it. Save only after validating the changed content.

`Name`	Select the relevant level first; read the adjacent controls/help in the editor before applying it. Save only after validating the changed content.
`First-time tip (shown once in the tip bar)`	Select the relevant level first; read the adjacent controls/help in the editor before applying it. Save only after validating the changed content.
`Bolt height`	Select the relevant level first; read the adjacent controls/help in the editor before applying it. Save only after validating the changed content.
`Difficulty (design)`	Select the relevant level first; read the adjacent controls/help in the editor before applying it. Save only after validating the changed content.
`Bolts (click a slot to paint; bottom = first nut)`	Select the relevant level first; read the adjacent controls/help in the editor before applying it. Save only after validating the changed content.
`+ Bolt`	Select the relevant level first; read the adjacent controls/help in the editor before applying it. Save only after validating the changed content.
`Remove bolt`	Select the relevant level first; read the adjacent controls/help in the editor before applying it. Save only after validating the changed content.
`<`	Select the relevant level first; read the adjacent controls/help in the editor before applying it. Save only after validating the changed content.
`>`	Select the relevant level first; read the adjacent controls/help in the editor before applying it. Save only after validating the changed content.
`Remove padlock`	Select the relevant level first; read the adjacent controls/help in the editor before applying it. Save only after validating the changed content.
`Difficulty curve (bar = 1 − casual bot win rate after Validate All, else the design value; green = breather, red = HARD)`	Select the relevant level first; read the adjacent controls/help in the editor before applying it. Save only after validating the changed content.
`Level number of the first (names)`	Select the relevant level first; read the adjacent controls/help in the editor before applying it. Save only after validating the changed content.

1. Exit Play mode. Back up `Assets/Games/NutSort/Resources/nutsort/Levels/levels.json`. Select an authored non-tutorial level to avoid unintentionally rewriting the onboarding sequence.
2. Use the available duplication/add control to make a working copy when offered. Change its display Name (or Picture name) and one gameplay property using the visual editor. Keep the first tutorial level intact until you understand its scripted guidance.
3. Run the editor's validation control and read the concrete warnings/errors. Fix missing references, invalid board geometry, impossible goals or malformed data before saving.
4. Save using the editor's Save control. If you intentionally edit JSON externally, save valid JSON, then use Reload/Reload from disk in the editor before continuing. Do not keep two unsaved versions of the same data open.
5. Use Play this level when available, or the documented DEV level navigation, to play the exact changed level. A solver/bot result is useful authoring evidence but does not replace checking the human controls, timing, tutorial and reward flow.

Procedural generation and level data reference

The level data used by this game is located here: `Assets/Games/NutSort/Resources/nutsort/Levels/levels.json`. Generation logic is in `Assets/Games/NutSort/Scripts/NsGen.cs`. Keep the authored campaign and generated continuation distinct when estimating your content count.

Generation controls exposed by the game editor include: `Difficulty curve (bar = 1 – casual bot win rate after Validate All, else the design value; green = breather, red = HARD)`, `Seed`, `Curve: values ramp linearly from the first to the last generated level`, `Generate`.

1. Back up the level JSON before a generation operation. Choose a small trial range and a repeatable seed first, then inspect what is appended or replaced. Do not assume a command is append-only; a selected-level regeneration or Rebuild command can replace authored content.
2. Read the game-specific fields in the level editor and generator source. Match generated board dimensions, rule limits and special mechanics to the intended difficulty; do not copy parameters from another game.
3. Validate generated output, play several early/mid/late boards, and check that the tutorial still introduces mechanics before the generated content requires them. Keep a saved known-good seed for regressions.
4. Treat visual difficulty labels and bot/solver scores as authoring aids. They do not guarantee that every generated board is suitable for every player or that a target completion time is achievable.

Authored field	Type / value in the included sample
`name`	`"First Turn"`
`cap`	`4`
`bolts`	`array of 2 entries`
`lockBolt`	`-1`
`keyBolt`	`-1`
`keySlot`	`-1`
`hard`	`false`
`breather`	`true`
`tut`	`1`
`hint`	`""`
`difficulty`	`0`

Use these field names exactly. JSON keys are serialized identifiers, not interface captions; changing a field name requires corresponding changes to the serializable model, loader, editor and generator. Duplicate levels through the editor when possible rather than guessing a nested schema.

Authoring controls and booster configuration

Actual control	Meaning
`Reload`	Reload saved JSON from disk; unsaved edits can be discarded.
`Undo`	Revert the last supported editor operation.
`Redo`	Reapply an undone editor operation.
`Validate`	Check the selected level and read the specific diagnostic.

`Validate All`	Check the whole campaign and inspect every failure.
`Add`	Create a new entry, configure it and validate before saving.
`Dup`	Duplicate the selected level before changing a working copy.
`Del`	Delete the selected entry; back up campaign data first.
`Up`	Move the selected level earlier in the campaign.
`Down`	Move it later in the campaign.
`Name`	Player-facing level name; do not rename the serialized field.
`Seed`	A repeatable random seed for reproducing a generated result.

Effect, unlock and first grant

The descriptions below are the actual game-authored booster help. Unlock and starting inventory are read from `Assets/Games/NutSort/Scripts/NutSortGame.cs`; existing saves keep their previously stored state.

Booster	Unlock level	Initial grant	Effect / game-authored help
Undo	3	3	Take back your last move.
Hint	5	2	Shows a good next move.
Spare Bolt	8	2	Screws an extra empty bolt onto the bench (once per level).
Rust Spray	10	2	Tap a bolt to clean off all its rust at once.

To rebalance, edit the matching `BoostUnlock` and `BoostStart` values and the corresponding handlers; keep the four entries in the same order. To rename or translate, edit `BoostName` in `Scripts/NutSortGame.cs` and `BoostDesc` in `Scripts/NutSortGame.cs`. A targeted booster may require a second tap on an eligible object; read its handler and test invalid targets before changing consumption logic. Refills must be granted only after the successful rewarded callback; cancelling an ad must not grant a reward.

Game text versus saved data and service configuration

Game progress, tutorial flags, currency and booster inventory are stored through the private `HzSave` helper and Unity PlayerPrefs. Search that helper and the main game script for the current key prefix. Changing displayed titles does not require changing those keys. Renaming keys can make an existing player appear to lose progress; plan migration before doing so.

Use a separate test profile when replaying onboarding. Do not use a global DeleteAll operation in a buyer's existing multi-asset project. Developer resets and unlocks should be used deliberately on test saves. Confirm the state after stopping/restarting Play, because an old saved tutorial flag can correctly suppress the popup you expect to see.

The HLK settings asset is `Assets/HyperCasualLiveOpsKit/Resources/HLK/HLKSettings.asset`. Ads default to Dummy and IAP to Disabled. Keep those defaults while reskinning. A production provider requires its real, separately installed SDK, IDs, platform configuration and testing. Optional setup documentation does not mean a provider SDK is included or automatically installed. Keep service identifiers, ad-placement IDs and reward callback semantics separate from translated button text.

Reskin troubleshooting and release check

Symptom	Check

A label reverts after one frame	Edit the refresh-time `.text =` assignment listed above, not only the creation argument. Search the exact visible phrase in Scripts, including portrait and landscape branches.
A scene edit disappears after stopping Play	The HUD is generated at runtime. Save the source/data edit outside Play mode, then enter a fresh Play session.
A colour changes on the HUD but not a popup	Inspect that popup's colour arguments and the separate popup palette symbols; it may use a different surface/ink pair or a baked bitmap.
A translated string clips	Preserve interpolation/line breaks, shorten the caption or intentionally adjust its Text/RectTransform size. Test the narrowest supported portrait layout and any longer language.
The HLK language table does not change the game label	That label uses a literal or game data. Edit its actual source/data or implement an explicit localization lookup there.
New art or audio is missing	Compare the exact resource base name, folder, importer settings and code lookup. Preserve its .meta file and verify there are no compile errors.
The old tutorial still does not appear	Check persisted tutorial/booster-first-use flags with a disposable save. Reaching an already-completed tutorial in an existing profile is not a localization failure.
The browser/Android demo looks unchanged	Rebuild and redeploy that demo after source changes. A hosted manual update does not rebuild the game.
Production ads or purchases are unavailable	This delivery uses simulated ads and Disabled IAP. Install/configure a supported real provider separately and test the complete result.

Before your own release: review branding, UI contrast, tutorial wording, authored/generated levels, saved progress, every booster, pause/mute controls and both layouts. Set your own application identifier, icons, version, signing and release privacy/service settings. Perform actual target-device/browser testing; this manual and its documentation-only update do not certify your customized binary.

Licences, support and documentation access

Keep the included font, icon, TextMesh Pro, audio and HyperCasual LiveOps Kit notices with your distribution. Marketing illustrations are separate from authentic gameplay screenshots. Refer to the included licence/AI disclosures for the supplied components; adding your own assets requires checking their distribution terms.

For product support, use the marketplace item support channel. SatisVizion support email: `satisvizion@gmail.com`. Include the game name, Unity version, target platform, exact Console error, changed files and clear reproduction steps. Do not send signing keys, passwords or private service credentials.

This is the main game manual. It is included offline in the main ZIP and the game's Documentation folder. Shared-kit detail is available in `HyperCasualLiveOpsKit.html` in the offline documentation bundle; within the Unity project, open the included kit's Documentation folder. The current online manual is linked in the publisher metadata. Installation and basic use do not depend on visiting an external storefront.